Solution: Good Number

Problem Statement

Write a Recursive Approach to Check if a Given Digit String Represents a Good Number.

A digit string is good if the digits (0-indexed) at even indices are even and the digits at odd indices are prime ((2, 3, 5, or 7).

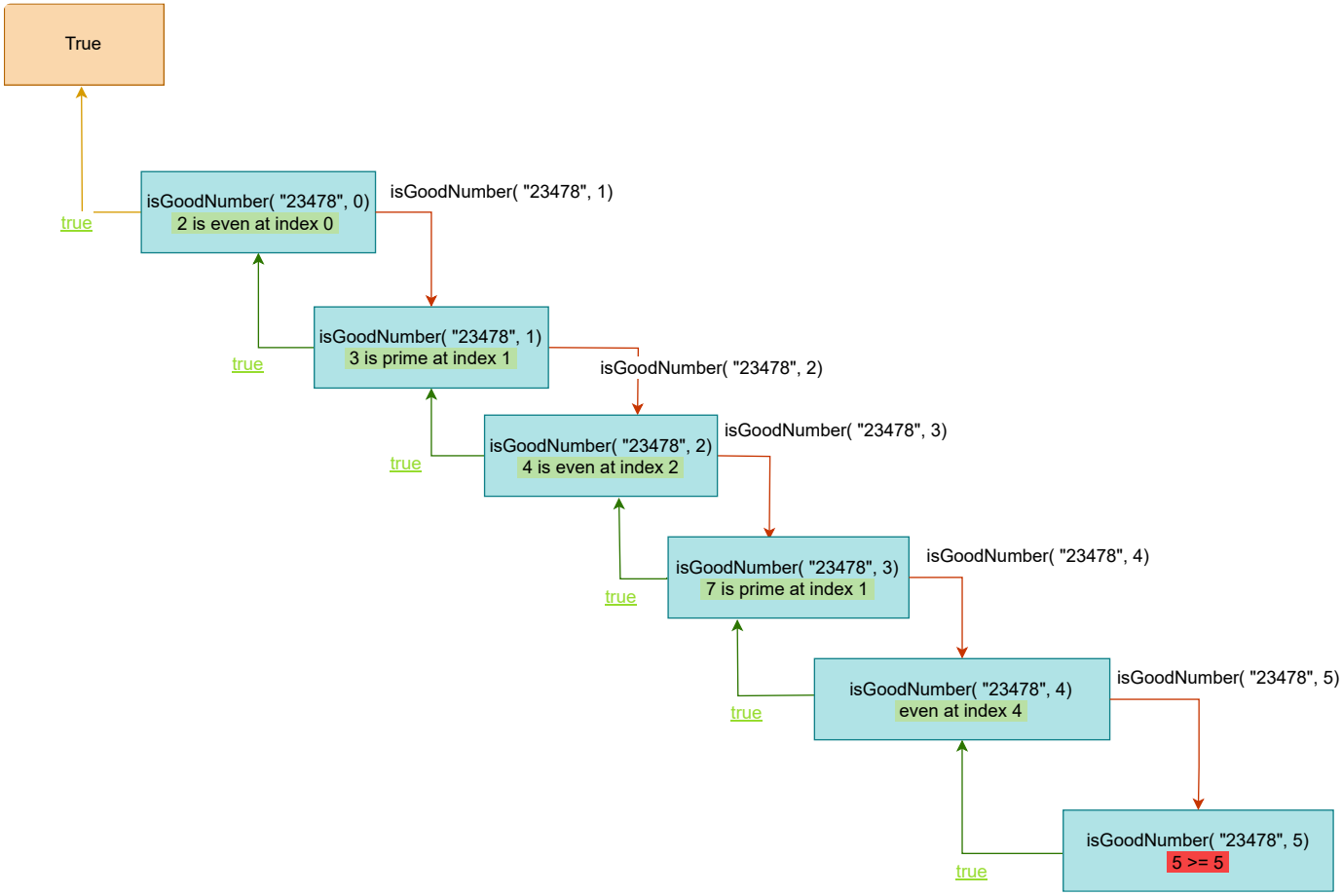
Examples:

Sr #	Input	Expected Output	Description
1	02468	Not Good	Number 6 at index 3 is not a prime number.
2	23478	Good	All digits at even indices are even, and all digits at odd indices are prime. Therefore, the digit string is considered "good".
3	224365	Good	All digits at even indices are even, and all digits at odd indices are prime. Therefore, the digit string is considered "good".

Solution:

1. The `isPrime` function is used to check if a number is prime.
2. The `isGoodNumber` function is a recursive function that checks each digit in the string. It takes two parameters: the digit string and the current index.
3. In the `isGoodNumber` function, the base case is when the index reaches the end of the string, in which case it returns `true`.
4. Inside the recursive function, it checks if the digit at the current index satisfies the conditions: even index digits must be even and odd index digits must be prime.
5. If the conditions are not met, it returns `false`.
6. Otherwise, it recursively calls `isGoodNumber` with the next index.
7. In the `main` function, it tests the `isGoodNumber` function on the example digit strings and prints the result.

This code uses recursion to traverse each digit in the string and checks if it satisfies the conditions for a "good number". The base case ensures that the recursion stops when all digits have been checked. By checking each digit recursively, the code can determine if the digit string is "good" or not.



Code

Here is the code for this algorithm:

Java

```java
import java.util.Arrays;
import java.util.List;

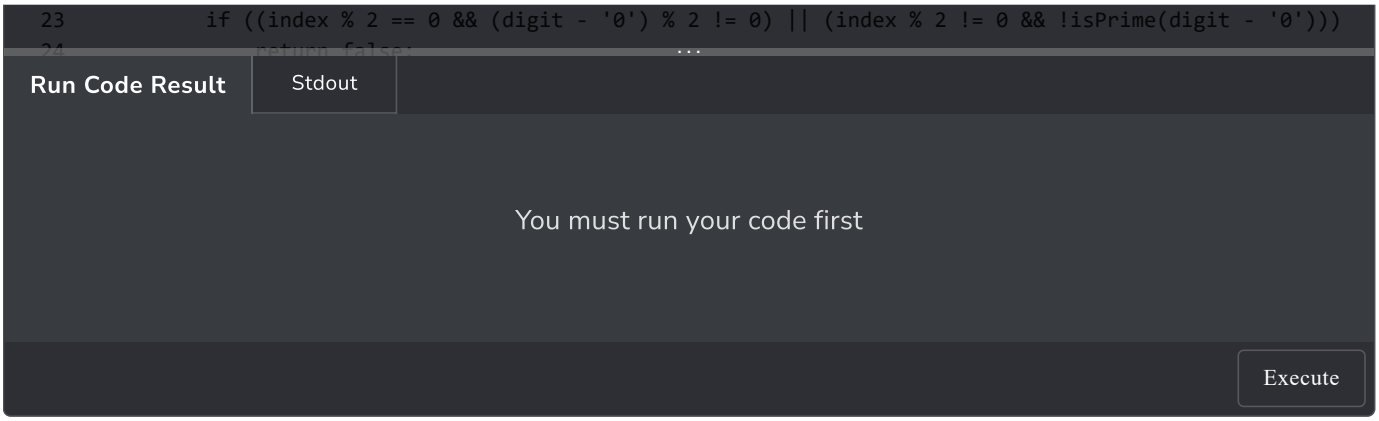
public class Solution {

    public static boolean isPrime(int num) {
        if (num < 2)
            return false;
        for (int i = 2; i * i <= num; i++) {
            if (num % i == 0)
                return false;
        }
        return true;
    }

    public static boolean isGoodNumber(String digits, int index) {
        if (index >= digits.length())
            return true;  // Base case: reached the end of the string

        char digit = digits.charAt(index);

        // Check if the digit at the current index satisfies the conditions
```



Space and Time Complexity

The time complexity of this algorithm is $O(N)$, where N is the length of the digit string. This is because the algorithm needs to check each digit in the string once. The space complexity is $O(N)$ as well, considering the recursive stack space used during the function calls.

← Previous

17. Good Number

Next →

18. Split BST

Mark as Completed